

Alegerea stack-ului tehnologic

1. TEHNOLOGIA PENTRU FRONTEND

1.1. Godot Engine 4

Frontend-ul jocului este dezvoltat utilizând Godot 4.5, un motor de joc open-source, modern și performant. Alegerea Godot este justificată de următoarele aspecte:

- suport excelent pentru jocuri 2D,
- integrare nativă cu WebSockets și WebRTC,
- arhitectură Node-Scene foarte accesibilă pentru prototipare rapidă,
- editor vizual intuitiv,
- performanță ridicată pe sisteme modeste,
- licență permisivă (MIT), fără costuri sau restricții
- permite exportul multiplatformă (Windows, Linux, macOS și Android).

2. TEHNOLOGIA PENTRU BACKEND

2.1. Node.js + TypeScript

Backend-ul este implementat în Node.js cu TypeScript, pentru a beneficia de:

- modelul asincron event-driven potrivit pentru aplicații realtime,
- latență redusă în gestionarea conexiunilor WebSocket,
- ecosistem matur (biblioteci, framework-uri, tooling),
- tipare stricte prin TypeScript pentru reducerea erorilor,
- scalare facilă (clustering, load balancing),
- integrare eficientă cu WebSockets (ws, uWebSockets),
- performanță excelentă pentru protocoale low-latency.

Această combinație este una dintre cele mai potrivite opțiuni pentru aplicații multiplayer.

3. TEHNOLOGIA PENTRU STOCAREA DATELOR

3.1. PostgreSQL

Baza de date utilizată este PostgreSQL, aleasă pentru:

- suport complet ACID, critic pentru tranzacții economice,
- suport nativ pentru JSONB (util pentru stocarea flexibilă a stărilor de joc),

- performanță ridicată la citire/scriere,
- mecanisme avansate de indexing,
- stabilitate și scalabilitate,
- compatibilitate excelentă cu ORM-uri TypeScript.

Pentru un proiect care manipulează resurse, tranzacții și indicatori economici, PostgreSQL reprezintă o bază robustă și matură.

4. LIMBAJUL ȘI FRAMEWORK-UL PRINCIPAL DE PROGRAMARE

4.1. GDScript (frontend)

GDScript este limbajul principal pentru partea de client, datorită:

- integrării native în Godot,
- sintaxei ușor de învățat,
- interoperabilității directe cu engine-ul,
- lipsa „boilerplate”-ului
- performanței bune pentru logică de joc 2D.

4.2. TypeScript (backend)

TypeScript este limbajul principal pentru logică server-side:

- tipare statice → cod mai sigur,
- debugging mai eficient,
- integrare cu Node.js,
- suport excelent pentru WebSockets și REST.

5. LIBRĂRII ȘI TEHNOLOGII UTILIZATE ÎN CADRUL PROIECTULUI

5.1. Frontend

- Godot WebSocket Client (modul nativ)
- Godot Animation System
- Godot UI Controls (GraphEdit, Containers, Panels)
- GDScript Signal System (pentru event-driven gameplay)

5.2. Backend

- ws (WebSocket server)

- Express.js (endpointuri REST)
- Prisma (ORM pentru PostgreSQL)
- jsonwebtoken (autentificare JWT)
- bcrypt (hashing parole)
- Node.js Cluster Mode (scalare orizontală posibilă)
- Zod / class-validator (validare input)

5.3. Tooling și utilitare

- Docker (containerizare DB și backend)
- pgAdmin (administrare DB)
- ESLint + Prettier (calitatea codului)
- Git + GitHub (control versiuni)

6. TOOL-URI DE DEVELOPMENT (IDE)

6.1. Frontend (Godot):

- Godot Editor 4.5 (instrument dedicat pentru dezvoltarea și testarea scenelor, scripturilor și interfețelor)

6.2. Backend (Node/TypeScript):

- Visual Studio Code
- WebStorm (**posibil**)
- extensii: ESLint, Prettier, GitLens, TypeScript Hero

6.3. Bază de date:

- pgAdmin 4
- DBeaver

7. JUSTIFICAREA ALEGERII TEHNOLOGIILOR ÎN DETRIMENTUL ALTERNATIVELOR

7.1. Godot vs. Unity / Unreal

- Godot este optim pentru 2D.
- Fără costuri, fără licențe suplimentare.
- Build rapid, editor foarte ușor de folosit.
- Integrare WebSocket directă și simplă.

7.2. Node.js + TypeScript vs. .NET Core

.NET Core este robust, dar:

- are overhead mai mare pentru scenarii realtime,
- ecosistem WebSocket mai complex,
- timpul de prototipare este mai mare,
- hosting-ul este mai scump.

Node.js este nativ orientat spre conexiuni simultane și mesaje dese, foarte potrivit pentru un joc multiplayer.

7.3. PostgreSQL vs. MySQL / MongoDB

- tranzații economice → necesită ACID,
- structuri complexe → în PostgreSQL permite combinarea relațional + JSON,
- performanță consistentă și foarte bună,
- suport avansat pentru locking și indexing.

MongoDB nu este potrivit pentru tranzații economice consistente. MySQL este limitat în JSON și indexing avansat.

7.4. TypeScript vs. JavaScript

- control strict al tipurilor,
- prevenirea bug-urilor,
- stabilitate în aplicații complexe,
- documentare mai clară a modelelor și entităților.